Metadata-as-Data in Apache Hudi: Multi-Modal Indexing and Concurrent Index Onboarding on Object Storage

First Author
Apache Hudi (PMC), Organization
City, Country
first.author@domain.com

Second Author
Organization
City, Country
second.author@domain.com

ABSTRACT

Open table formats increasingly serve as the transactional layer for lakehouse architectures built on cloud object storage. Yet, object stores expose costly metadata operations (e.g., recursive listings) and make planning-time statistics expensive to obtain at scale. Apache Hudi addresses these bottlenecks by treating metadata and indexes as first-class *table data*: each table maintains an internal *metadata table* (an internal Merge-on-Read table) that stores file listings and multiple index modalities (column statistics, bloom filters, record/secondary/expression indexes) in an SSTable-like layout. To avoid blocking high-throughput ingestion, Hudi further supports *concurrent index onboarding*—a two-phase (plan/execute) protocol that backfills large indexes asynchronously while writers continue to commit. We describe this "metadata-as-data" design and its concurrency model, and we summarize representative results on TPC-DS where a secondary index reduces scanned data by 90% and improves warm-run latency by 58%. We conclude with interoperability implications: index/metadata interfaces remain inconsistent across engines, motivating standardized contracts for index discovery, consumption, and online backfill in open table formats.

KEYWORDS

Lakehouse, table formats, object storage, metadata management, data skipping, adaptive indexing, Apache Hudi

1 INTRODUCTION

The lakehouse pattern relies on open file formats (e.g., Parquet) and table formats (log-structured tables) to provide ACID semantics, schema evolution, and time travel over cloud object stores [2, 3]. However, as tables scale to millions of objects, *metadata* becomes a dominant cost. Object stores implement a key–value interface where directory listings and bulk footer/stat reads can be expensive; Delta Lake explicitly calls out costly object-store metadata operations (such as listing) as a barrier to high performance [4]. Meanwhile, once metadata grows large, it requires database-style management techniques [5].

This paper focuses on one architectural response in Apache Hudi: **metadata-as-data**. Hudi maintains an internal *metadata table* that stores file listings and multiple index modalities, enabling lookup-based planning and pruning in place of repeated recursive listings and broad footer scans [6]. Further, Hudi supports **concurrent index onboarding** so that large, expensive indexes can be backfilled asynchronously without pausing ingestion [7]. These mechanisms directly align with the FORMATS workshop interest in metadata organization, indexing, and workload-aware maintenance [1].

Diagram placeholder. Engines obtain candidate files via the `files` partition (metadata-based listing) and prune using `column_stats`/`partition_stats`. Equality predicates can consult `record_index` and optional secondary/expression index partitions.

Figure 1: Metadata-as-data: file listings and multi-modal indexes stored in Hudi's metadata table.

Contributions. We contribute: (1) a compact description of Hudi's metadata table design as an internal, scalable index substrate; (2) an explanation of Hudi's multi-modal indexing as metadata-table partitions and its online backfill protocol; and (3) a small, representative evaluation showing query pruning benefits from secondary indexing on TPC-DS.

2 METADATA TABLE AS AN INDEX SUBSTRATE

2.1 Overview

Hudi tracks metadata about a table to remove bottlenecks in achieving read/write performance, especially on cloud storage. In particular, it (i) avoids list operations to obtain the set of files in a table and (ii) exposes column statistics to reduce expensive broad footer reads for planning and pruning [6]. Each Hudi table contains a special internal *metadata table*, implemented as a single internal Merge-on-Read (MoR) table, with different metadata/index types stored as partitions [6]. For physical storage, the metadata table uses HFile as the base file format to support efficient key lookups [6].

2.2 Metadata types and access patterns

The `files` partition stores file listings (names, sizes, active state) and the set of partitions, enabling metadata-based listing without direct object-store listings [6]. The `column_stats` and `partition_stats` partitions store min/max/null-counts and related statistics, enabling data skipping during query planning [6]. A `bloom_filters` partition stores bloom filters centrally to avoid scanning footers across many data files [8].

2.3 Multi-modal indexing as metadata partitions

Hudi extends the metadata table with additional partitions to implement *multi-modal indexing*. A record index maintains a mapping from record keys to file locations to support fast upserts/deletes and point lookups, while secondary indexes map non-key column values to record keys and use the record index to retrieve file locations [8]. Expression indexes index transformed values (e.g.,

Table 1: TPC-DS (1TB) secondary index benchmark on `web_sales` (Spark 3.5.5, Hudi 1.0.1). Results from [9].

Setting	Run 1 (s)	Run 2 (s)	GB read	Files read
No secondary index	32	14	67	5000
With secondary index	22	6	7	521

`from_unixtime(ts)`) to optimize predicates that do not directly match raw columns [8]. Hudi's indexing documentation describes this as enhancing the metadata table with new index types as partitions, complemented by an asynchronous index-building mechanism [8].

3 CONCURRENT INDEX ONBOARDING

A central tradeoff in indexing is balancing write throughput with index maintenance. Hudi supports creating indexes via SQL and programmatic APIs, including asynchronous indexing, to avoid locking out writers on large tables [7]. For very large tables, Hudi provides **concurrent indexing** using a *two-phase protocol* [7]: (i) *scheduling/planning* creates an indexing plan over all completed commits up to an indexing instant under a lock, and (ii) *execution* builds index files and then validates/catches up commits that completed during execution under a metadata-table lock. This design mixes optimistic concurrency control with multi-version techniques so other writers are unblocked while an index snapshot is established [7].

4 EVALUATION: SECONDARY INDEX ON TPC-DS

We report a representative benchmark from Hudi's secondary index evaluation on a 1TB TPC-DS dataset [9]. The setup used Spark 3.5.5 on an EMR cluster and Hudi 1.0.1, building a secondary index on the `web_sales` fact table (`ws_ship_customer_sk`) and running a join query with a point predicate on that column [9]. Table 1 summarizes the published results: the secondary index reduced scanned data from 67 GB to 7 GB (90%) and reduced warm-run latency from 14 s to 6 s (58%), while reducing files read from 5,000 to 521 [9].

While this experiment isolates one index type, it demonstrates the key benefit of metadata-resident indexes for "needle-in-a-haystack" predicates. Complementary results in Hudi's indexing deep dive report that, on a 400 GB synthetic table with 20,000 file groups, a point predicate on a single record key improved from 977 s to 12 s when using the record index [10].

5 INTEROPERABILITY IMPLICATIONS

Storing metadata efficiently is necessary but not sufficient: engines must *consume* it. Hudi documents that Spark and Flink can use metadata-based listing; Presto can fetch file names/sizes from the metadata table; and support in Trino for reading from the metadata table was dropped in Trino 419 [6]. Amazon Athena notes that it currently supports only the file listing index from Hudi's metadata table [11]. These discrepancies suggest an ecosystem gap: we lack standardized contracts for index discovery and safe consumption across engines.

We argue that open table formats should standardize: (i) schemas for common index modalities (file listings, stats, row/secondary/expression indexes), (ii) an online backfill protocol (like Hudi's plan/execute) with well-defined consistency, and (iii) capability negotiation so engines can safely leverage available indexes.

6 CONCLUSION

Hudi's "metadata-as-data" pattern stores file listings and multi-modal indexes inside an internal metadata table to avoid object-store metadata bottlenecks and enable lookup-based pruning. Hudi further supports concurrent index onboarding to build expensive indexes asynchronously without pausing ingestion. Our summarized evaluation shows that metadata-resident secondary indexing can reduce scanned data by 90% and improve warm-run latency by 58% on TPC-DS. More broadly, interoperability remains the limiting factor: to fully realize metadata-rich formats across heterogeneous compute and emerging AI/vector workloads, the community needs standardized index/metadata interfaces and online backfill semantics.

REFERENCES

[1] FORMATS'26. Call for Papers: International Workshop on Data Formats. https://dataformats.org/call-for-papers.html.

[2] Michael Armbrust, Ali Ghodsi, Reynold Xin, and Matei Zaharia. Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics. In *CIDR*, 2021.

[3] Jesús Camacho-Rodríguez, Ashvin Agrawal, Anja Gruenheid, Ashit Gosalia, Cristian Petculescu, Josep Aguilar-Saborit, Avrilia Floratou, Carlo Curino, and Raghu Ramakrishnan. LST-Bench: Benchmarking Log-Structured Tables in the Cloud. *Proc. ACM Manag. Data*, 2(1), 2024.

[4] Michael Armbrust et al. Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores. *Proc. VLDB Endow.*, 13(12):3411–3424, 2020.

[5] Pavan Edara and Mosha Pasumansky. Big Metadata: When Metadata is Big Data. *Proc. VLDB Endow.*, 14(12):3083–3095, 2021.

[6] Apache Hudi. Table Metadata (Documentation). https://hudi.apache.org/docs/metadata/.

[7] Apache Hudi. Indexing / Concurrent Indexing (Documentation). https://hudi.apache.org/docs/metadata_indexing/.

[8] Apache Hudi. Indexes (Documentation). https://hudi.apache.org/docs/indexes/.

[9] Dipankar Mazumdar and Aditya Goenka. Introducing Secondary Index in Apache Hudi Lakehouse Platform. Apache Hudi Blog, April 2, 2025. https://hudi.apache.org/blog/2025/04/02/secondary-index/.

[10] Shiyan Xu. Deep Dive Into Hudi's Indexing Subsystem (Part 2 of 2). Apache Hudi Blog, November 12, 2025. https://hudi.apache.org/blog/2025/11/12/deep-dive-into-hudis-indexing-subsystem-part-2-of-2/.

[11] Amazon Athena. Use Hudi metadata for improved performance. https://docs.aws.amazon.com/athena/latest/ug/querying-hudi-metadata-table.html.